

P2374R4

views::cartesian_product

Published Proposal, 2022-07-13

This version:

<http://wg21.link/P2374>

Authors:

Sy Brand (sy.brand at microsoft dot com)

Michał Dominiak (griwes at griwes dot info)

Audience:

LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes `std::ranges::cartesian_product_view` for taking the cartesian product of multiple forward ranges.

Table of Contents

1 Changelog

- 1.1 Changes from r3
- 1.2 Changes from r2
- 1.3 Changes from r1
- 1.4 Changes from r0

2 Motivation

3 Design

- 3.1 Minimum Range Requirements
- 3.2 Tuple or pair
- 3.3 [reference_type](#)
- 3.4 Empty cartesian product view
- 3.5 Common Range
- 3.6 Bidirectional Range
- 3.7 Random Access Range
- 3.8 Initial Range Argument Special-Casing
- 3.9 Sized Range
- 3.10 Naming
- 3.11 Pipe Support
- 3.12 Borrowed Range?

4 Implementation

5 Wording

- 5.1 Addition to `<ranges>`
- 5.2 Range adaptor helpers [`range.adaptor.helpers`]
- 5.3 Cartesian product view [`range.cartesian`]
 - 5.3.1 Overview [`range.cartesian.overview`]

- 5.3.2 Class template `cartesian_product_view` [range.cartesian.view]
- 5.3.3 Class template `cartesian_product_view::iterator` [ranges.cartesian.iterator]
- 5.4 Feature-test macro

6 Acknowledgements

§ 1. Changelog

§ 1.1. Changes from r3

LWG review comments:

- Fixed various typos and applied minor comments.
- Fix fold expressions to be uniform between the forward declaration and the definition; add parentheses to fold expressions where previously missed.
- Removed constraints from the `default_sentinel_t` overload of `end`; made it `noexcept`.
- Reworded the specification of `size` to have a precondition that the return value actually fits in the return type, and to make it less redundant.
- Reformatted the document to correctly italicize exposition only names and provide paragraph numbers in the specification.

LWG review comments, second round:

- The non-sentinel overloads of `end` no longer call `begin` more than once on the first range.
- Add a void version of `operator++(int)`, and constrain the non-void one.
- A lot of the `maybe-const` invocations from the iterator definition have been pulled into the exposition-only concepts. This does *not* apply to: `cartesian-product-common-arg`, because it's used by the other concepts and by an exposition-only function template; and `cartesian-product-is-common` and `cartesian-product-is-sized`, as these are always used with a known constness of all of their arguments.
- Applied an equivalent of the resolution of LWG3692 to this paper, removing the relational operators in iterator and removing spurious constraints from spaceship.
- `cartesian-sentinel-is-sized` has been turned into `cartesian-is-sized-sentinel`, and it now accepts a class template that is used to select the type for the checked "sentinel" type for the first range. During the wording review the authors were asked to make this concept take a pair of iterator/sentinel types for the first range, but that would require the sites of use to still use `maybe-const` on the first range type *twice*; making this a concept that also accepts an argument that is either `iterator_t` or `sentinel_t` in its usage makes this simpler while also achieving the goal of simplifying the specification of the subtraction operator between two iterators.
- The definition of `distance-to` has been reformatted, and the expressions it uses are now math expressions to avoid overflow issues and allowing to specify a precondition correctly. `scaled_distance` used to have a couple of spurious arguments; this was a leftover from previous specification attempts and has been fixed.
- The definition of `operator+=` has been rewritten in terms of hypothetical iterator values instead of "as-if `prev/next` was called X times". These hypothetical iterator values are now also used to define a precondition that the resulting iterator must contain only iterators that are within the underlying ranges.
- Various small fixes to make sure constraints, semantics, and formatting are correct.

LWG review comments, mailing list review:

- Fixed the handling of a cartesian product with an empty non-first range in comparison with `default_sentinel_t` in the non-common case.
- The new description of `cartesian_product_view` claimed "any number" of ranges, but the view type itself (as opposed to `views::cartesian_product`) takes a non-zero number of ranges. This has been fixed.
- Various typo fixes.

LWG review comments, third round:

- Rename `distance-to` to `distance-from` to avoid semantic confusion with `std::distance`.

- Add a missing "Returns: *this." to `operator+=`.
- Rephrase the precondition of `operator+=` once again, this time in terms of hypothetical values of calls to `ranges::distance`.
- Replace "for every N in the interval [x, y]" with "for every integer $x \leq N \leq y$ " throughout the paper.
- Reword the exception specification of `iter_move` to be in the style of the exception specification of `iter_swap`.
- Various typo and formatting fixes.

§ 1.2. Changes from r2

- Make the size and difference types implementation-defined and only recommend requirements for them.

§ 1.3. Changes from r1

- Add wording adding `cartesian_product_view` into the synopsis of `<ranges>`.
- Made the constructor of the view `explicit`.
- Added a design section about not making `cartesian_product_view` a borrowed range.
- Added a feature test macro.
- Use the verbiage about the return type of `size` to also define the iterator's `difference_type`.
- Relax the requirements on the first range, allowing it to be an input range.
- Relax the requirements on the ranges for the view to be a common range - it now only depends on the properties of the first range.
- Add `operator-` overloads that allow computing the distance between an iterator and a sentinel.
- Various wording fixes.

§ 1.4. Changes from r0

- Specify the return type of `size`.
- Add design section on the first range argument.

§ 2. Motivation

`Cartesian product` is a fundamental mathematical construct. There should be a `cartesian_product_view` which generates the cartesian product of any number of ranges, e.g.:

Before	After
<pre>std::vector<int> a,b,c; for (auto&& ea : a) { for (auto&& eb : b) { for (auto&& ec : c) { use(ea,eb,ec); } } }</pre>	<pre>std::vector<int> a,b,c; for (auto&& [ea,eb,ec] : std::views::cartesian_product(a,b,c)) { use(ea,eb,ec); }</pre>

This is especially useful for composing with other views, or dealing with parameter packs of ranges:

Before	
<pre>template <std::size_t N = 0, class F, class Res, class Tuple, class... Args> auto find_tuples_satisfying_impl(</pre>	<pre>template <class F, std::ranges::for requires(std::regular_invocable<F, std::ranges::range_refere</pre>

```

    F f, Res& res, Tuple const& ranges, Args const&... args)
requires(N == tuple_size_v<std::remove_cvref_t<Tuple>>()) {
    if (std::invoke(f, args...)) {
        res.push_back(std::make_tuple(args...));
    }
}

template <std::size_t N = 0, class F,
          class Res, class Tuple, class... Args>
auto find_tuples_satisfying_impl(
    F f, Res& res, Tuple const& ranges, Args const&... args) {
    for (auto&& e : std::get<N>(ranges)) {
        find_tuples_satisfying_impl<N+1>(f, res, ranges, args..., e);
    }
}

template <class F, std::ranges::forward_range... Vs>
requires(std::regular_invocable<
    F, std::ranges::range_reference_t<Vs>...>)
auto find_tuples_satisfying(F f, Vs&&... vs) {
    std::vector<tuple<std::ranges::range_value_t<Vs>...>> res;
    find_tuples_satisfying_impl(
        f, res, tuple(std::forward<Vs&&>(vs)...));
    return res;
}

```

```

auto find_tuples_satisfying(F f, Vs
    return std::views::cartesian_prod
        | std::views::filter([f](auto&&
        | std::ranges::to<std::vector>(
    }

```

§ 3. Design

§ 3.1. Minimum Range Requirements

This paper requires all ranges, except the first one, passed to [cartesian_product_view](#) to be forward ranges. See [Initial Range Argument Special-Casing](#) for discussion on the first argument being an input range.

§ 3.2. Tuple or pair

A potential design would be to use [tuple](#) as the [value_type](#) and [reference_type](#) of [cartesian_product_view](#). This paper uses `std::pair` if two ranges are passed and [tuple](#) otherwise. See [p2321](#) for motivation.

§ 3.3. [reference_type](#)

This paper uses `tuple-or-pair<ranges::reference_t<Vs>...>` as the reference type. See [p2214](#) for discussion of value types (particularly [pair](#) and [tuple](#)) as the [reference_type](#) for ranges, and [p2321](#) for wording of improvements for key use-cases.

§ 3.4. Empty cartesian product view

Trying to take the cartesian product view of 0 views will produce an `empty_view<tuple<>>`, in parity with Range-v3 and [p2321](#).

§ 3.5. Common Range

[cartesian_product_view](#) can be a common range if the first range either is common, or is sized and random access. This paper reflects this.

§ 3.6. Bidirectional Range

`cartesian_product_view` can be a bidirectional range if the underlying ranges, except for the first one, are bidirectional and common, or if they are random access and sized. Non-common bidirectional ranges are not supported because decrementing when one of the iterators is at the beginning of its range would require advancing the iterator to end, which may be linear in complexity.

We don't consider non-common, random access, sized ranges as worth supporting, so this paper requires bidirectional and common.

§ 3.7. Random Access Range

`cartesian_product_view` can be a random access range if all the underlying ranges are random access and sized, with the second requirement not applying to the first range. Sized ranges are required because when the view is incremented, the new states of the iterators are calculated modulo the size of their views.

We can't think of too many use-cases for this and it adds a fair bit of implementation burden, but this paper supports the necessary operations.

§ 3.8. Initial Range Argument Special-Casing

The first range passed to `cartesian_product_view` can be treated specially, since it is only passed through a single time. Therefore, the specification relaxes several constraints on the first range passed:

- It could be an input range instead of forward range
- It wouldn't need to be a sized range in order for the `cartesian_product_view` to be random-access or common
- It wouldn't need to be common in order for the `cartesian_product_view` to be bidirectional

Previous revisions of this paper didn't propose this feature, however the Ranges SG has requested that it be added to the paper.

§ 3.9. Sized Range

`cartesian_product_view` can be a sized range if all the underlying ranges are, in which case the size is the product of all underlying sizes. This is reflected in the paper.

§ 3.10. Naming

An alternative name is `std::ranges::product_view`. This paper uses `cartesian_product_view` as we believe it is more explicit in what its semantics are.

§ 3.11. Pipe Support

It may be possible to support syntax such as `vec1 | views::cartesian_product(vec2)` by either fixing the number of arguments allowed to the view, or adding a pipe operator to `cartesian_product_view` itself.

However, it's problematic for the same reason as `views::zip`, in that one cannot distinguish between the usages in `a | views::cartesian_product(b, c)` and `views::cartesian_product(b, c)` on its own. As such, this paper does not support this syntax.

§ 3.12. Borrowed Range?

It is possible to implement `cartesian_product_view` in such a way that it is a borrowed range if all the underlying ranges are borrowed. However, this requires that every iterator of the view stores two iterators per every range used for the cartesian product - one for the current position within that range, and another for the begin iterator, to allow wrapping

around back to the first element when the end of that range is reached. (For random access underlying ranges, it is conceivable to store an iterator and an offset, but this is not applicable in general.) If `cartesian_product_view` is not a borrowed view, the iterators only require storing one iterator per every range, and a pointer to the view itself.

Due to this, this paper does not make `cartesian_product_view` a borrowed range.

§ 4. Implementation

There are implementations of a cartesian product view in `Range-v3`, `cor3ntin::rangesnext`, and `tl::ranges`, among others.

§ 5. Wording

§ 5.1. Addition to `<ranges>`

Add the following to 24.2 [ranges.syn], Header `<ranges>` synopsis:

```
// ...
namespace std::ranges {
    // ...

    // [range.cartesian], cartesian product view
    template<input_range First, forward_range... Vs>
        requires (view<First> && ... && view<Vs>)
    class cartesian_product_view;

    namespace views { inline constexpr unspecified cartesian_product = unspecified; }
}
```

§ 5.2. Range adaptor helpers [range.adaptor.helpers]

Add a new section after Non-propagating cache [range.nonprop.cache]. Remove the definitions of `tuple-or-pair`, `tuple-transform`, and `tuple-for-each` from Class template `zip_view` [range.zip.view] and add them to this new section:

```
namespace std::ranges {
    template <class... Ts>
        using tuple-or-pair = see-below; // exposition only

    template<class F, class Tuple>
    constexpr auto tuple-transform(F&& f, Tuple&& tuple) { // exposition only
        return apply([&<class... Ts>(Ts&&... elements) {
            return tuple-or-pair<invoke_result_t<F&, Ts>...>(
                invoke(f, std::forward<Ts>(elements))...
            );
        }, std::forward<Tuple>(tuple));
    }

    template<class F, class Tuple>
    constexpr void tuple-for-each(F&& f, Tuple&& tuple) { // exposition only
        apply([&<class... Ts>(Ts&&... elements) {
            (invoke(f, std::forward<Ts>(elements)), ...);
        }, std::forward<Tuple>(tuple));
    }
}
```

Given some pack of types Ts, the alias template `tuple-or-pair` is defined as follows:

1. If `sizeof...(Ts)` is 2, `tuple-or-pair<Ts...>` denotes `pair<Ts...>`.

2. Otherwise, `tuple-or-pair<Ts...>` denotes `tuple<Ts...>`.

§ 5.3. Cartesian product view [range.cartesian]

§ 5.3.1. Overview [range.cartesian.overview]

1. `cartesian_product_view` takes any non-zero number of ranges n and produces a view of tuples calculated by the n -ary cartesian product of the provided ranges.
2. The name `views::cartesian_product` denotes a customization point object. Given a pack of subexpressions `Es...`, the expression `views::cartesian_product(Es...)` is expression-equivalent to
 - `decay-copy(views::empty<tuple<>>)` if `Es` is an empty pack,
 - otherwise, `cartesian_product_view<views::all_t<decltype((Es))>...>(Es...)`.

[Example:

```
std::vector<int> v { 0, 1, 2 };
for (auto&& [a,b,c] : std::views::cartesian_product(v, v, v)) {
    std::cout << a << ' ' << b << ' ' << c << '\n';
    //0 0 0
    //0 0 1
    //0 0 2
    //0 1 0
    //0 1 1
    //...
}
```

-- end example]

§ 5.3.2. Class template `cartesian_product_view` [range.cartesian.view]

```
namespace std::ranges {
    template <bool Const, class First, class... Vs>
    concept cartesian-product-is-random-access = // exposition only
        (random_access_range<maybe-const<Const, First>> && ... &&
         (random_access_range<maybe-const<Const, Vs>>
          && sized_range<maybe-const<Const, Vs>>));

    template <class R>
    concept cartesian-product-common-arg = // exposition only
        common_range<R> || (sized_range<R> && random_access_range<R>);

    template <bool Const, class First, class... Vs>
    concept cartesian-product-is-bidirectional = // exposition only
        (bidirectional_range<maybe-const<Const, First>> && ... &&
         (bidirectional_range<maybe-const<Const, Vs>>
          && cartesian-product-common-arg<maybe-const<Const, Vs>>));

    template <class First, class... Vs>
    concept cartesian-product-is-common = // exposition only
        cartesian-product-common-arg<Const, First>;

    template <class... Vs>
    concept cartesian-product-is-sized = // exposition only
        (sized_range<Vs> && ...);

    template<bool Const, template<class> class FirstSent, typename First, typename... Vs>
    concept cartesian-is-sized-sentinel = // exposition only
        (sized_sentinel_for<FirstSent<maybe-const<Const, First>,
         iterator_t<maybe-const<Const, First>> && ...
         && (sized_range<maybe-const<Const, Vs>>
```

```

        && sized_sentinel_for<iterator_t<maybe-const<Const, Vs>>,
            iterator_t<maybe-const<Const, Vs>>>>));

template <cartesian-product-common-arg R>
constexpr auto cartesian-common-arg-end(R & r) {
    if constexpr (common_range<R>) {
        return ranges::end(r);
    }
    else {
        return ranges::begin(r) + ranges::distance(r);
    }
}

template <input_range First, forward_range... Vs>
    requires (view<First> && ... && view<Vs>)
class cartesian_product_view
: public view_interface<cartesian_product_view<First, Vs...>> {
private:
    tuple<First, Vs...> bases_; // exposition only

    template<bool Const>
    struct iterator; // exposition only
public:
    constexpr cartesian_product_view() = default;
    constexpr explicit cartesian_product_view(First first_base, Vs... bases);

    constexpr iterator<false> begin()
        requires (!simple-view<First> || ... || !simple-view<Vs>);
    constexpr iterator<true> begin() const
        requires (range<const First> && ... && range<const Vs>);

    constexpr iterator<false> end()
        requires ((!simple-view<First> || ... || !simple-view<Vs>) &&
            cartesian-product-is-common<First, Vs...>);
    constexpr iterator<true> end() const
        requires cartesian-product-is-common<const First, const Vs...>;
    constexpr default_sentinel_t end() const noexcept;

    constexpr see below size()
        requires cartesian-product-is-sized<First, Vs...>;
    constexpr see below size() const
        requires cartesian-product-is-sized<const First, const Vs...>;
};

template <class... Vs>
cartesian_product_view(Vs&&...) -> cartesian_product_view<all_t<Vs>...>;

namespace views { inline constexpr unspecified cartesian_product = unspecified; }
}

```

```
constexpr explicit cartesian_product_view(First first_base, Vs... bases);
```

1. *Effects*: Initialises `bases_` with `std::move(first_base)`, `std::move(bases)...`.

```
constexpr iterator<false> begin()
    requires (!simple-view<First> || ... || !simple-view<Vs>);
```

2. *Effects*: Equivalent to: `return iterator<false>(tuple-transform(ranges::begin, bases_));`

```
constexpr iterator<true> begin() const
    requires (range<const First> && ... && range<const Vs>);
```


3. *Effects*: Equivalent to: `return iterator<true>(tuple-transform(ranges::begin, bases_));`

```
constexpr iterator<false> end()
    requires (!!simple-view<First> || ... || !simple-view<Vs>)
        && cartesian-product-is-common<First, Vs...>;
constexpr iterator<true> end() const
    requires cartesian-product-is-common<const First, const Vs...>;
```

4. *Let*:

- `is-const` be `true` for the `const`-qualified overload, and `false` otherwise;
- `is-empty` be `true` if the expression `ranges::empty(rng)` is `true` for any `rng` among the underlying ranges except the first one and `false` otherwise; and
- `begin-or-first-end(rng)` be expression-equivalent to `is-empty ? ranges::begin(rng) : cartesian-common-arg-end(rng)` if `rng` is the first underlying range and `ranges::begin(rng)` otherwise.

5. *Effects*: Equivalent to:

```
iterator<is-const> it(tuple-transform(
    [](auto & rng){ return begin-or-first-end(rng); }, bases_));
return it;
```

```
constexpr default_sentinel_t end() const noexcept;
```

6. *Returns*: `default_sentinel`.

```
constexpr see below size()
    requires cartesian-product-is-sized<First, Vs...>;
constexpr see below size() const
    requires cartesian-product-is-sized<const First, const Vs...>;
```

7. The return type is an implementation-defined unsigned-integer-like type.

8. *Recommended practice*: The return type should be the smallest unsigned-integer-like type that is sufficiently wide to store the product of the maximum sizes of all the underlying ranges, if such a type exists.

9. Let p be the product of the sizes of all the ranges in `bases_`.

10. *Preconditions*: p can be represented by the return type.

11. *Returns*: p .

§ 5.3.3. Class template `cartesian_product_view::iterator` [ranges.cartesian.iterator]

```
namespace std::ranges {
    template<input_range First, forward_range... Vs>
    requires (view<First> && ... && view<Vs>)
    template<bool Const>
    class cartesian_product_view<First, Vs...>::iterator {
    public:
        using iterator_category = input_iterator_tag;
        using iterator_concept = see below;
        using value_type = tuple-or-pair<range_value_t<maybe-const<Const, First>>,
            range_value_t<maybe-const<Const, Vs>>...>;
        using reference = tuple-or-pair<reference_t<maybe-const<Const, First>>,
            reference_t<maybe-const<Const, Vs>>...>;
        using difference_type = see below;

        iterator() requires forward_range<maybe-const<Const, First>> = default;

        constexpr iterator(iterator<!Const> i) requires Const &&
```

```

    (convertible_to<iterator_t<First>, iterator_t<maybe-const<Const, First>>> &&
      ... && convertible_to<iterator_t<Vs>, iterator_t<maybe-const<Const, Vs>>>);

constexpr auto operator*() const;
constexpr iterator& operator++();
constexpr void operator++(int);
constexpr iterator operator++(int) requires forward_range<maybe-const<Const, First>>;

constexpr iterator& operator--()
    requires cartesian-product-is-bidirectional<Const, First, Vs...>;
constexpr iterator operator--(int)
    requires cartesian-product-is-bidirectional<Const, First, Vs...>;

constexpr iterator& operator+=(difference_type x)
    requires cartesian-product-is-random-access<Const, First, Vs...>;
constexpr iterator& operator-=(difference_type x)
    requires cartesian-product-is-random-access<Const, First, Vs...>;

constexpr reference operator[](difference_type n) const
    requires cartesian-product-is-random-access<Const, First, Vs...>;

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<iterator_t<maybe-const<Const, First>>>;

friend constexpr bool operator==(const iterator& x, default_sentinel_t);

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires all-random-access<Const, First, Vs...>;

friend constexpr iterator operator+(const iterator& x, difference_type y)
    requires cartesian-product-is-random-access<Const, First, Vs...>;
friend constexpr iterator operator+(difference_type x, const iterator& y)
    requires cartesian-product-is-random-access<Const, First, Vs...>;
friend constexpr iterator operator-(const iterator& x, difference_type y)
    requires cartesian-product-is-random-access<Const, First, Vs...>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires cartesian-is-sized-sentinel<Const, iterator_t, First, Vs...>;

friend constexpr difference_type operator-(iterator i, default_sentinel_t)
    requires cartesian-is-sized-sentinel<Const, sentinel_t, First, Vs...>;
friend constexpr difference_type operator-(default_sentinel_t, iterator i)
    requires cartesian-is-sized-sentinel<Const, sentinel_t, First, Vs...>;

friend constexpr auto iter_move(const iterator& i) noexcept(see below);

friend constexpr void iter_swap(const iterator& l, const iterator& r) noexcept(see below)
    requires (indirectly_swappable<iterator_t<maybe-const<Const, First>>> && ... &&
      indirectly_swappable<iterator_t<maybe-const<Const, Vs>>>);

private:
    maybe-const<Const, cartesian_product_view>* parent_ = nullptr; // exposition only
    tuple-or-pair<iterator_t<maybe-const<Const, First>>,
      iterator_t<maybe-const<Const, Vs>>...> current_; // exposition only

    template <size_t N = sizeof...(Vs)>
    constexpr void next(); // exposition only

    template <size_t N = sizeof...(Vs)>
    constexpr void prev(); // exposition only

    template <class Tuple>
    constexpr difference_type distance-from(Tuple t); // exposition only

    constexpr explicit iterator(tuple-or-pair<iterator_t<maybe-const<Const, First>>,

```

```

        iterator_t<maybe-const<Const, Vs>>...> current); // exposition only
};
}

```

1. `iterator::iterator_concept` is defined as follows:

- If `cartesian-product-is-random-access<Const, First, Vs...>` is modeled, then `iterator_concept` denotes `random_access_iterator_tag`.
- Otherwise, if `cartesian-product-is-bidirectional<Const, First, Vs...>` is modeled, then `iterator_concept` denotes `bidirectional_iterator_tag`.
- Otherwise, if `maybe-const<Const, First>` models `forward_range`, then `iterator_concept` denotes `forward_iterator_tag`.
- Otherwise, `iterator_concept` denotes `input_iterator_tag`.

2. `iterator::difference_type` is an implementation-defined signed-integer-like type.

3. *Recommended practice:* `iterator::difference_type` should be the smallest signed-integer-like type that is sufficiently wide to store the product of the maximum sizes of all underlying ranges if such a type exists.

```

template <size_t N = sizeof...(Vs)>
constexpr void next(); // exposition only

```

4. *Effects:* Equivalent to:

```

auto& it = std::get<N>(current_);
++it;
if constexpr (N > 0) {
    if (it == ranges::end(get<N>(parent_->bases_))) {
        it = ranges::begin(std::get<N>(parent_->bases_));
        next<N - 1>();
    }
}

```

```

template <size_t N = sizeof...(Vs)>
constexpr void prev(); // exposition only

```

5. *Effects:* Equivalent to:

```

auto& it = std::get<N>(current_);
if (it == ranges::begin(std::get<N>(parent_->bases_))) {
    it = cartesian-common-arg-end(std::get<N>(parent_->bases_));
    if constexpr (N > 0) {
        prev<N - 1>();
    }
}
--it;

```

```

template <class Tuple>
constexpr difference_type distance-from(Tuple t); // exposition only

```

6. Let:

- `scaled_size(N)` be the product of `static_cast<difference_type>(ranges::size(std::get<N>(parent_->bases_)))` and `scaled_size(N+1)` if $N < \text{sizeof} \dots (Vs)$, otherwise `static_cast<difference_type>(1)`;
- `scaled_distance(N)` be the product of `static_cast<difference_type>(std::get<N>(current_) - std::get<N>(t))` and `scaled_size(N+1)`; and
- `scaled_sum` be the sum of `scaled_distance(N)` for every integer $0 \leq N \leq \text{sizeof} \dots (Vs)$.

7. *Preconditions*: `scaled_sum` can be represented by `difference_type`.

8. *Returns*: `scaled_sum`.

```
constexpr explicit iterator(tuple-or-pair<iterator_t<maybe-const<Const, First>>,
    iterator_t<maybe-const<Const, Vs>>...> current);
```

9. *Effects*: Initializes `current_` with `std::move(current)`.

```
constexpr iterator(iterator<!Const> i) requires Const &&
    (convertible_to<iterator_t<First>, iterator_t<maybe-const<Const, First>>>> &&
     ... && convertible_to<iterator_t<Vs>, iterator_t<maybe-const<Const, Vs>>>>);
```

10. *Effects*: Initializes `current_` with `std::move(i.current_)`.

```
constexpr auto operator*() const;
```

11. *Effects*: Equivalent to:

```
return tuple-transform([](auto& i) -> decltype(auto) { return *i; }, current_);
```

```
constexpr iterator& operator++();
```

12. *Effects*: Equivalent to:

```
next();
return *this;
```

```
constexpr void operator++(int);
```

13. *Effects*: Equivalent to `++*this`.

```
constexpr iterator operator++(int) requires forward_range<maybe-const<Const, First>>>;
```

14. *Effects*: Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

```
constexpr iterator& operator--()
    requires cartesian-product-is-bidirectional<Const, First, Vs...>;
```

15. *Effects*: Equivalent to:

```
prev();
return *this;
```

```
constexpr iterator operator--(int)
    requires cartesian-product-is-bidirectional<Const, First, Vs...>;
```

16. *Effects*: Equivalent to:

```
auto tmp = *this;
--*this;
return tmp;
```

```
constexpr iterator& operator+=(difference_type x)
    requires cartesian-product-is-random-access<Const, First, Vs...>;
```

17. Let `orig` be the value of `*this` before the call.

18. Let `ret` be:

- If $x > 0$, the value of `*this` had `next` been called x times.
- Otherwise, if $x < 0$, the value of `*this` had `prev` been called $-x$ times.
- Otherwise, `orig`.

19. *Precondition*: x is in the range `[ranges::distance(*this, ranges::begin(*parent_)), ranges::distance(*this, ranges::end(*parent_))]`.

20. *Effects*: Sets the value of `*this` to `ret`.

21. *Returns*: `*this`.

22. *Complexity*: Constant.

```
constexpr iterator& operator-=(difference_type x)
    requires cartesian-product-is-random-access<Const, First, Vs...>;
```

23. *Effects*: Equivalent to:

```
*this += -x;
return *this;
```

```
constexpr reference operator[](difference_type n) const
    requires cartesian-product-is-random-access<Const, First, Vs...>;
```

24. *Effects*: Equivalent to: `return *((*this) + n);`

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<iterator_t<maybe-const<Const, First>>>
```

25. *Effects*: Equivalent to: `return x.current_ == y.current_;`

```
friend constexpr bool operator==(const iterator& x, default_sentinel_t);
```

26. *Returns*: true if `std::get<i>(x.current_) == ranges::end(std::get<i>(x.parent_>->bases_))` is true for any integer $0 \leq i \leq \text{sizeof} \dots (\text{Vs})$; otherwise, false.

```
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires all-random-access<Const, First, Vs...>;
```

27. *Effects*: Equivalent to: `return x.current_ <= y.current_;`

```
friend constexpr iterator operator+(const iterator& x, difference_type y)
    requires cartesian-product-is-random-access<Const, First, Vs...>;
```

28. *Effects*: Equivalent to: `return iterator(x) += y;`

```
friend constexpr iterator operator+(difference_type x, const iterator& y)
    requires cartesian-product-is-random-access<Const, First, Vs...>;
```

29. *Effects*: Equivalent to: `return y + x;`

```
friend constexpr iterator operator-(const iterator& x, difference_type y)
    requires cartesian-product-is-random-access<Const, First, Vs...>;
```

30. *Effects*: Equivalent to: `return iterator(x) -= y;`

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires cartesian-is-sized-sentinel<Const, iterator_t, First, Vs...>;
```

31. *Effects*: Equivalent to: `return x.distance-from(y.current_);`

```
friend constexpr difference_type operator-(iterator i, default_sentinel_t)
    requires cartesian-is-sized-sentinel<Const, sentinel_t, First, Vs...>;
```

32. Let `end_tuple` be an object of a type that is a specialization of `tuple`, such that:

- `std::get<0>(end_tuple)` has the same value as `ranges::end(std::get<0>(i.parent_>bases_))`;
- `std::get<N>(end_tuple)` has the same value as `ranges::begin(std::get<N>(i.parent_>bases_))` for every integer $1 \leq N \leq \text{sizeof} \dots (Vs)$.

33. *Effects*: Equivalent to: `return i.distance-from(end_tuple);`

```
friend constexpr difference_type operator-(default_sentinel_t s, iterator i)
    requires cartesian-is-sized-sentinel<Const, sentinel_t, First, Vs...>;
```

34. *Effects*: Equivalent to: `return -(i - s);`

```
friend constexpr auto iter_move(const iterator& i) noexcept(see below);
```

35. *Effects*: Equivalent to: `return tuple-transform(ranges::iter_move, i.current_);`

36. *Remarks*: The exception specification is equivalent to the logical AND of the following expressions:

- `noexcept(ranges::iter_move(std::get<N>(i.current_))` for every integer $0 \leq N \leq \text{sizeof} \dots (Vs)$,
- `is_nothrow_move_constructible_v<range_rvalue_reference_t<maybe-const<Const, T>>>` for every type `T` in `F, Vs...`.

```
friend constexpr void iter_swap(const iterator& l, const iterator& r) noexcept(see below)
    requires (indirectly_swappable<iterator_t<maybe-const<Const, First>>> && ... &&
        indirectly_swappable<iterator_t<maybe-const<Const, Vs>>>);
```

37. *Effects*: For every integer $0 \leq i \leq \text{sizeof} \dots (Vs)$, performs: `ranges::iter_swap(std::get<i>(l.current_), std::get<i>(r.current_))`.

38. *Remarks*: The exception specification is equivalent to the logical AND of the following expressions:

`noexcept(ranges::iter_swap(std::get<i>(l.current_), std::get<i>(r.current_)))` for every integer $0 \leq i \leq \text{sizeof} \dots (Vs)$.

§ 5.4. Feature-test macro

Add the following macro definition to 17.3.2 [version.syn], Header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_ranges_cartesian_product 20XXXXL // also in <ranges>
```

§ 6. Acknowledgements

Thank you to Christopher Di Bella, Corentin Jabot, Tim Song, and Barry Revzin for feedback and guidance.

Thank you to Tomasz Kamiński for help with getting the wording into a good shape.